

Checking if you're asked to reply in non-technical way for ADHD reader?

10.0s

Yes. I'm set to reply in plain, non-technical language for an ADHD reader: next action first, numbered steps, progress restated, no preamble or tangents.

What would you like me to do?

Here's my task

What changed recently?

i think our stop hook or some mechanism is broken. why is claude not stopped from replying in TUI; it should be asked to reply in agent chat only



1m 39s

The stop hook is real and running — but it has a blind spot, and that's why the TUI-only reply slipped through.

What the hook is supposed to do: block a turn from ending unless I sent something you can see (send_message / send_progress).

Why it failed on "What changed recently?":

1. The hook figures out "where does this turn start?" by scanning for your last typed message. It deliberately skips lines marked as tool results.
2. When you reply by clicking a quick-reply (or answering a blocking prompt), your message does NOT arrive as a fresh typed line — it comes back as the *return value* of my send_message call, which is recorded as a tool result. So the hook skips it.
3. That means the hook draws the turn boundary too far back — at your earlier typed message ("check_messages; reply me..."), not at "What changed recently?".
4. Inside that too-wide window, an *earlier* turn's send_message is still sitting there. The hook sees it, decides "a visible message already happened," and lets the turn end.
5. So my "What changed recently?" answer, written as plain terminal text with no send_message, was never caught.

In short: after your first typed message, every follow-up reply in this chat arrives as a tool result, which the hook ignores when finding the turn start — so it counts an old reply as covering a new turn.

This is a fixable bug in the hook's turn-boundary logic, not something you did.

Want me to fix the hook?

Yes, fix the hook

No, leave it for now

Show me the exact fix first

56.8s

One block changes in /home/app/.claude/hooks/swe-swe-stop-guard.sh — the part that decides where the current turn starts.

Now (broken):

```
# Slice the transcript from the last real user message. Tool results also
# arrive as type:user lines; excluding them keeps the slice to this turn.
n=$(grep -n 'P0 :P1 ' "$tp" | grep -v tool_result | tail -1 | cut -d: -f1)
[ -n "$n" ] || exit 0
```

This throws away every tool-result line. But your chat replies ARE tool-result lines, so it anchors on your oldest typed message and the window swallows a previous turn's send.

Fix (replace that block with):

```
# Slice from the last GENUINE user message. Typed messages are type:user
# non-tool_result lines. Agent-chat replies arrive as a send_message /
# check_messages RESULT (type:user + tool_result) whose text is P0
# or P1 . Anchor on either -- otherwise back-to-back chat turns
# share one anchor and an earlier turnP7 /P2 :P3 / {
    if (!/"tool_result"/ || /User said:|User responded:/) last = NR
} END { if (last) print last }' "$tp")
[ -n "$n" ] || exit 0
```

What it does: it now treats a chat reply (a tool result that literally says "User said:" / "User responded:") as a real turn start. So the window begins at your latest message, no old send is inside it, and a plain-terminal-only turn gets blocked as intended.

Why it's safe: the only new lines it can anchor on are ones containing "User said:" / "User responded:". If some other command output happened to contain that text, the effect is a *smaller* window — which means *more* blocking, never less. It can't weaken the guard.

Nothing else in the hook changes. The empty-queue silent-turn exception still works.

Apply it?

Not yet

Yes, apply it

2m 9s

Done. The hook is fixed and tested.

What I changed: one block in /home/app/.claude/hooks/swe-swe-stop-guard.sh — it now recognizes your chat replies (which arrive as tool results saying "User said:" / "User responded:") as real turn starts, so the turn window no longer reaches back and counts an old message-send.

Proof (4 tests):

1. A turn that answers only in the terminal with no send → now BLOCKED. (Before: it slipped through — the bug.)
2. A turn that does send a chat message → passes.
3. A "check for messages, queue empty" quiet turn → still allowed.
4. The one-nudge-then-let-it-pass escape → still works.

No commit needed: this hook lives in your machine's home config, not in the project's version-controlled files.

One caveat I left alone (pre-existing, unrelated to this bug): the hook confirms "a message was sent" by text-searching the log for the send command's name. If some unrelated command output happens to print that exact name, it could be mistaken for a real send. It didn't cause your issue and fixing it is a separate, larger change.

Want me to tackle that caveat too?

No, that's all

Yes, fix the caveat too

Explain the caveat more

discuss how would that be tackled

33.1s

The fix: stop searching the log as plain text; read it as structured data and only count a send that was *actually performed*.

Why that works: the log records each tool the assistant runs as its own labeled entry. A genuine send shows up as a real "I ran the send tool" entry. Text that merely spells out the tool's name is just words in some output — it has no such entry. Reading the structure tells the two apart; searching for the name can't.

Steps to build it:

1. Replace the name-search with a structured read (using jq , a tool already required by the hook — it walks the log's data fields).
2. Count a send only if there's a real "ran a send tool" entry after your latest message.
3. Handle the two ways a send can happen: the built-in send tool, and the command-line version run through a shell. For the shell version, check only inside that command's own text (a tight, safe spot), not the whole log.
4. Cover the siblings too: progress updates, drawings, voice replies.
5. Add tests: a real send passes; a real command-line send passes; a fake log line that only prints the tool name is rejected; progress/drawing variants pass.

Effort: small-to-moderate — roughly a 6-line rewrite plus a handful of test cases. About 30–45 minutes.

Is it worth it? Low urgency. Triggering the fake-pass needs a command that prints the exact tool name — which basically only happens when someone is debugging the hook itself (what I did earlier). It's real, but rare, and it never affected your normal use.

My recommendation: worth doing as quick hardening since the change is small and well-contained, but not urgent — fine to defer.

Do it now, or leave it?

Leave it for now

Just write it up as a task

Do it now

3m 35s

Done. The caveat is fixed and tested.

What changed: the hook no longer decides "a message was sent" by searching the log text for the tool's name. It now reads the log as structured data and requires a real record that the send tool was actually run.

Two ways a send is recognized:

1. The built-in send tool — now airtight; a genuine "ran the tool" record, impossible to fake with printed text.
2. The command-line send (rarely used) — only counted when the send command sits at the real start of a command, not when its name appears as an argument to something else.

Tested 9 cases, all correct:

1. Terminal-only answer, no send → blocked.
2. Built-in send → passes.
3. Progress update → passes.
4. Command-line send at command start → passes.
5. Command-line send chained after "&&" → passes.
6. Send name used only as a search argument → blocked (the tightening).
7. Fake log line that merely prints the tool name → blocked (the original spoof, now closed).
8. "Queue empty" quiet check → still allowed.
9. A broken/garbled log line sitting before a real send → real send still detected.

No commit: still a machine-level config file, not in the project's tracked files.

Residual (honest note): the command-line send path is still recognized by text (there's no clean way around that without fully parsing shell commands), but it's now tightly scoped and that path is effectively unused here. The normal built-in send path has no such gap.

Anything else?

No, that's all

Something else

what's the full file path changed in this session ?

5.2s

The code file I changed:

```
/home/app/.claude/hooks/swe-swe-stop-guard.sh
```

That's the hook. I also touched two note-keeping files (not code):

1. /home/app/.claude/projects/-repos-form-is-type-workspace/memory/project_stop_guard_is_type_toolresult_gap.md
2. /home/app/.claude/projects/-repos-form-is-type-workspace/memory/MEMORY.md

All three are in your machine's home config, not in the project's tracked files.

Anything else?

No, that's all

Something else

Type a message...

Send